

Guía Práctica: Instalación y Configuración de PostgreSQL en Windows

Esta guía está diseñada para estudiantes de la Especialización en Análítica de Datos e Inteligencia de Negocios. Cubrirá desde la instalación estándar para cualquier usuario hasta configuraciones avanzadas orientadas a proyectos de ciencia de datos utilizando entornos virtuales (Conda y archivos .env).

Parte 1: Instalación Estándar de PostgreSQL en Windows

Paso 1: Descarga del Instalador

1. Ingresa al sitio web oficial de descargas: [PostgreSQL Downloads for Windows](#).
2. Haz clic en el enlace "**Download the installer**".
3. Selecciona la versión más estable y reciente compatible con Windows (se recomienda **PostgreSQL 16 o 17**) y descarga el archivo ejecutable (.exe).
- 4.

Paso 2: Proceso de Instalación

1. Ejecuta el archivo descargado como **Administrador** (clic derecho -> *Ejecutar como administrador*).
2. Haz clic en *Next* en la ventana de bienvenida.
3. **Directorio de instalación:** Deja la ruta por defecto (ej. C:\Program Files\PostgreSQL\<versión>) y haz clic en *Next*.
4. **Selección de componentes:** Asegúrate de marcar los siguientes elementos (por defecto vienen seleccionados):
 - ☒ **PostgreSQL Server** (El motor de la base de datos).
 - ☒ **pgAdmin 4** (La interfaz gráfica oficial para administrar la base de datos).
 - ☒ **Stack Builder** (Opcional, útil para instalar controladores adicionales).
 - ☒ **Command Line Tools** (Herramientas de consola como psql).
5. **Directorio de datos:** Deja la ruta predeterminada para almacenar las bases de datos y haz clic en *Next*.
6. **Contraseña del superusuario (postgres):**
 - Introduce una contraseña segura (ej. admin123 o una clave personal robusta).
 - **⚠ IMPORTANTE:** Anota esta contraseña, ya que la necesitarás para conectar cualquier script de Python o cliente GUI a la base de datos.
7. **Puerto:** Deja el puerto por defecto **5432** (a menos que ya tengas otra instancia ocupándolo). Haz clic en *Next*.
8. **Configuración regional:** Selecciona *Default locale* y haz clic en *Next*.
9. Revisa el resumen de la instalación y haz clic en **Next** para iniciar la copia de archivos. Al

finalizar, desmarca la opción de *Stack Builder* si aparece y haz clic en *Finish*.

Paso 3: Verificación del Entorno (Variables de Sistema)

Para poder ejecutar comandos de PostgreSQL desde cualquier terminal:

1. Abre el menú de inicio de Windows y busca "**Variables de entorno**".
2. Selecciona "*Editar las variables de entorno del sistema*" y haz clic en el botón *Variables de entorno...*
3. En la sección *Variables del sistema*, busca la variable llamada **Path** y haz clic en *Editar*.
4. Haz clic en *Nuevo* y añade la ruta de los binarios de PostgreSQL (por lo general: C:\Program Files\PostgreSQL\<versión>\bin).
5. Haz clic en *Aceptar* en todas las ventanas abiertas.
6. Abre una nueva terminal (CMD o PowerShell) y escribe:

```
PS C:\Windows> psql --version
psql (PostgreSQL) 17.6
```

Si te devuelve la versión instalada, el entorno está listo.

Parte 2 (Avanzado): Configuración de Base de Datos en Entornos Virtuales con Conda y Archivos .env

En proyectos profesionales de analítica de datos e inteligencia de negocios, las credenciales de acceso a las bases de datos **nunca** se escriben directamente en el código fuente (por seguridad). Se gestionan mediante variables de entorno aisladas utilizando entornos virtuales de Conda y archivos .env.

Paso 0: Instalando [conda](#)

Para instalar conda, primero debes elegir el instalador adecuado. A continuación, se muestran los instaladores más populares disponibles actualmente:

→ [Miniconda](#)

Miniconda es un instalador mínimo proporcionado por Anaconda. Utilice este instalador si desea instalar la mayoría de los paquetes manualmente.

→ [Distribución Anaconda](#)

Anaconda Distribution es un instalador completo que incluye un conjunto de paquetes para la ciencia de datos, así como Anaconda Navigator, una aplicación con interfaz gráfica de usuario para trabajar con entornos conda.

→ [Miniforja](#)

Miniforge es un instalador mantenido por la comunidad conda-forge que viene preconfigurado para usarse con el canal conda-forge. Para obtener más información sobre conda-forge, visite [su sitio web](#).

Paso 1: Creación y Activación del Entorno Virtual en Conda

Abre tu terminal (Anaconda Prompt o terminal de VS Code) y ejecuta los siguientes comandos para crear un entorno limpio enfocado en analítica:

Bash

```
# 1. Crear un entorno virtual llamado 'analitica_env' con Python
conda create -n analitica_env python=3.10 -y
```

2. Activar el entorno virtual

```
conda activate analitica_env
```

3. Instalar las librerías necesarias para conexión a PostgreSQL (psycopg2) y análisis de datos (pandas, sqlalchemy)

```
conda install pandas sqlalchemy psycopg2 -y
```

Paso 2: Creación del Archivo de Configuración .env

En la raíz de la carpeta de tu proyecto de Python, crea un archivo de texto plano llamado exactamente **.env** (sin extensiones ocultas como .txt).

Dentro de este archivo, escribe las credenciales de tu servidor de PostgreSQL local o remoto:

Fragmento de código

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=proyecto_analitica_db
DB_USER=postgres
DB_PASSWORD=tu_contraseña_segura
```

Nota: Asegúrate de que la base de datos proyecto_analitica_db haya sido creada previamente en tu pgAdmin o mediante consola.

Paso 3: Carga de Variables de Entorno y Conexión desde Python

Para leer de forma segura las credenciales del archivo .env dentro de tu script o Jupyter Notebook, necesitas instalar la librería python-dotenv:

Bash

```
pip install python-dotenv
```

A continuación, implementa el siguiente script de prueba en Python para verificar la conexión utilizando **SQLAlchemy** y **Pandas**:

Python

```
import os
import pandas as pd
from dotenv import load_dotenv
from sqlalchemy import create_engine

# 1. Cargar las variables definidas en el archivo .env
load_dotenv()
```

```

# 2. Leer las variables de entorno de manera segura
DB_HOST = os.getenv("DB_HOST")
DB_PORT = os.getenv("DB_PORT")
DB_NAME = os.getenv("DB_NAME")
DB_USER = os.getenv("DB_USER")
DB_PASSWORD = os.getenv("DB_PASSWORD")

# 3. Construir la cadena de conexión (Connection String) para SQLAlchemy
# Formato: postgresql+psycopg2://usuario:contraseña@host:puerto/nombre_bd
connection_string =
f"postgresql+psycopg2://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}"

try
    # 4. Crear el motor de conexión
    engine = create_engine(connection_string)

    # 5. Probar la conexión ejecutando una consulta simple con Pandas
    query = "SELECT version();"
    df_version = pd.read_sql(query, con=engine)

    print(f"¡Conexión exitosa a PostgreSQL utilizando el entorno virtual y archivo .env!")
    print(df_version.iloc[0, 0])

except Exception as e:
    print(f"Error al conectar con la base de datos: {e}")

```

Ventajas de este enfoque avanzado para tu Proyecto:

- **Seguridad:** Evitas exponer claves de acceso en repositorios de código abiertos como GitHub.
- **Portabilidad:** Si compartes tu proyecto con compañeros de equipo (ingenieros o administradores), cada quien configura su propio archivo .env localmente apuntando a sus credenciales sin alterar el código analítico.
- **Integración con Pandas:** Facilita la extracción directa de datos tabulares desde PostgreSQL hacia DataFrames para su posterior modelado y visualización.